Moteur de Tarification Réassurance

Documentation des fonctions clés

`severity.py` `frequency.py` `reinsurance.py`

1 Présentation de l'application

1.1 Contexte

Une compagnie d'assurance supporte chaque année un portefeuille de sinistres dont la charge totale est aléatoire et potentiellement très élevée lors d'années catastrophiques. Pour se protéger, elle transfère une partie de ce risque à un **réassureur** via un *traité de réassurance*, en échange d'une prime.

L'application web est un **outil d'aide à la décision**.

1. **Quelle est la structure de mon risque ?**
   L'outil ajuste automatiquement des lois statistiques sur les données historiques de sinistres (fréquence et sévérité), identifiant la distribution qui représente le mieux le comportement du portefeuille.
2. **Quel sera le coût de mes sinistres dans le futur ?**
   À partir des lois ajustées, un moteur Monte-Carlo génère des milliers d'années synthétiques de sinistres, produisant une distribution complète de la charge annuelle.
3. **Quel traité choisir et à quel prix ?**
   L'utilisateur configure un ou plusieurs traités (Quote-Part, Excédent de sinistre) et l'outil calcule les métriques de risque avant et après réassurance — VaR, TVaR, probabilité de ruine ainsi que la prime théorique selon différents principes actuariels.

1.2 Architecture des données

Les sinistres sont séparés en deux populations aux comportements distincts :

Population	Définition	Modélisation typique
Attritionnelle (*below*)	Sinistres courants, sous un seuil	Loi légère (Gamma, LogNormale...)
Grave (*above*)	Grands sinistres, au-dessus du seuil	Loi à queue lourde

Chaque population possède sa propre loi de fréquence (nombre de sinistres/an) et de sévérité (montant par sinistre), ajustées indépendamment.

1.3 Flux de traitement global

1. Ajustement	→	**2. Simulation**	→	**3. Tarification**
Données historiques		N années fictives		Prime & indicateurs de risque
`severity.py` / `frequency.py`		`reinsurance.py`		`reinsurance.py`

2 Module `severity.py` — Ajustement de la sévérité

2.1 Principe

Pour chaque population, plusieurs lois candidates sont ajustées et celle ayant le meilleur critère d'information (AIC/BIC) est retenue. Les lois testées couvrent un spectre allant des distributions à queue modérée (`Gamma`, `LogNormale`, `Weibull`) aux distributions à queue lourde adaptées aux grands sinistres.

2.2 `safe_fit_distribution(montants_sinistres, nom_loi)`

Ajuste une loi par maximum de vraisemblance et calcule les critères de sélection de modèle.

Algorithm 1 Ajustement d'une distribution par maximum de vraisemblance

Entrée : `montants_sinistres` : vecteur des montants historiques, `nom_loi` : loi à tester
Sortie : dictionnaire {paramètres estimés, log-vraisemblance, AIC, BIC}

1: `params_estimes` $\leftarrow$ MLE(`montants_sinistres`, `nom_loi`)
2: `log_vraisemblance` $\leftarrow \sum_i \log f_{\texttt{nom_loi}}(\texttt{montant}_i \mid \texttt{params_estimes})$
3: **if** `log_vraisemblance` non fini ou optimisation échouée **then return** `None`
4: **end if**
5: `nb_params` $\leftarrow$ nombre de paramètres dans `params_estimes`
6: `nb_sinistres` $\leftarrow$ taille de `montants_sinistres`
7: **AIC** $\leftarrow 2 \cdot \texttt{nb_params} - 2 \cdot \texttt{log_vraisemblance}$
8: **BIC** $\leftarrow$ `nb_params` $\cdot \ln(\texttt{nb_sinistres}) - 2 \cdot$ `log_vraisemblance` **return** {`params_estimes`, `log_vraisemblance`, AIC, BIC}

2.3 `compute_gof_stats(montants_sinistres, resultat_ajustement, nom_loi)`

Mesure la qualité de l'ajustement via deux tests statistiques complémentaires.

Algorithm 2 Tests de qualité d'ajustement

Entrée : `montants_sinistres`, `params_estimes`, `nom_loi`

1: `montants_tries` $\leftarrow$ tri croissant de `montants_sinistres`
2: `cdf_theorique` $\leftarrow F_{\texttt{nom_loi}}(\texttt{montants_tries} \mid \texttt{params_estimes})$
3: `stat_KS`, `pval_KS` $\leftarrow$ test de Kolmogorov-Smirnov ▷ écart max entre CDF empirique et théorique
4: `stat_AD` $\leftarrow$ statistique d'Anderson-Darling ▷ pénalise davantage les écarts en queue **return** `stat_KS`, `stat_AD`, `pval_KS`

3 Module `frequency.py` — Ajustement de la fréquence

3.1 Principe

On compte le nombre de sinistres dépassant le seuil par année d'observation, puis on ajuste trois lois de comptage candidates.

Loi	Paramètres estimés	Usage typique
`Poisson`	$\lambda = \bar{n}$	Variance $\approx$ moyenne
`Binomiale Négative`	r, p	Sur-dispersion ($\sigma^2 > \mu$)
`Géométrique`	$p = 1/(1+\bar{n})$	Très forte variabilité annuelle

3.2 `compute_counts_from_dates(dates_sinistres, masque_seuil, annee_debut)`

Transforme une série de dates de sinistres bruts en comptages annuels exploitables.

Algorithm 3 Construction des comptages annuels à partir des dates

Entrée : `dates_sinistres`, `masque_seuil` : booléen (sinistre au-dessus du seuil ?), `annee_debut`

1: Filtrer : garder les lignes où `date valide` ET `masque_seuil` ET `annee` $\geq$ `annee_debut`
2: `comptages_par_annee` $\leftarrow$ regrouper les sinistres filtrés par année civile
3: Compléter les années sans sinistre avec la valeur 0 **return** `comptages_par_annee`, `liste_annees`

3.3 `analyze_frequency(comptages_annuels)`

Algorithm 4 Sélection de la meilleure loi de fréquence

Entrée : `comptages_annuels` : vecteur du nombre de sinistres par année
Sortie : dictionnaire des résultats (AIC, BIC, paramètres) pour chaque loi testée

1: **for** `nom_loi` $\in$ {Poisson, Binomiale Négative, Géométrique} **do**
2: `params_estimes` $\leftarrow$ MLE sur `comptages_annuels`
3: Calculer `log_vraisemblance`, `AIC`, `BIC`
4: **if** `log_vraisemblance` est fini **then**
5: Stocker le résultat pour `nom_loi`
6: **end if**
7: **end forreturn** tous les ajustements valides

4 Module `reinsurance.py` — Simulation et tarification

4.1 `simuler_depuis_distributions(nb_annees_simulees, ...)`

Génère N années synthétiques de sinistres via Monte-Carlo entièrement vectorisé.

Algorithm 5 Simulation Monte-Carlo vectorisée

Entrée : `nb_annees_simulees` N, lois fréquence et sévérité pour *below* et *above*
Sortie : `matrice_below` $\in \mathbb{R}^{N\times k_b}$, `matrice_above` $\in \mathbb{R}^{N\times k_a}$ (0 = pas de sinistre)

1: `nb_sinistres_below_par_annee` $\leftarrow$ tirer N fréquences en un seul appel
2: `total_sinistres_below` $\leftarrow \sum$ `nb_sinistres_below_par_annee`
3: `tous_montants_below` $\leftarrow$ tirer `total_sinistres_below` sévérités en un seul appel
4: Répartir `tous_montants_below` dans `matrice_below` selon `nb_sinistres_below_par_annee`
5: Répéter les étapes 1–4 pour la population *above* $\rightarrow$ `matrice_above` **return** {`matrice_below`, `matrice_above`}

4.2 `_apply_traite_matrix(matrice_sinistres, traite)`

Applique un traité sur l'ensemble des années simulées en une seule opération matricielle.

Quote-Part (QP) — L'assureur conserve un pourcentage fixe τ de chaque sinistre :

$$\texttt{montant_net} = \tau \times \texttt{montant_brut}$$

Excédent de sinistre (XS) — L'assureur cède la tranche entre la priorité P et le plafond $P + L$:

$$\texttt{montant_cede} = \min\big(\max(\texttt{montant_brut} - P,\ 0),\ L\big), \quad \texttt{montant_net} = \texttt{montant_brut} - \texttt{montant_cede}$$

Algorithm 6 Application d'un traité XS sur toute la matrice (sans boucle)

Entrée : `matrice_sinistres` $(N \times k)$, `priorite` P, `portee` L

1: `matrice_cessions` $\leftarrow \min\big(\max(\texttt{matrice_sinistres} - P, 0), L\big)$ **return** `matrice_sinistres` $-$ `matrice_cessions`

4.3 `compute_premium(cessions_simulees_par_annee, principe, parametre)`

Calcule la prime de réassurance à partir du vecteur des cessions simulées.

Principe	Formule	Paramètre
Valeur espérée	$P_R = (1+\theta)\,\mathbb{E}[\texttt{cessions}]$	θ : chargement
Écart-type	$P_R = \mathbb{E}[\texttt{cessions}] + \alpha\,\sigma_{\texttt{cessions}}$	α
Variance	$P_R = \mathbb{E}[\texttt{cessions}] + \alpha\,\mathrm{Var}(\texttt{cessions})$	α

4.4 `compute_full_stats(...)`

Fonction principale — orchestre toute la chaîne et retourne les métriques de risque pour les trois vues.

Algorithm 7 Calcul des statistiques complètes brut / cédé / net

Entrée : simulations, `liste_traites`, `principe_prime`, `capital_disponible`

1: `charges_brutes_par_annee` $\leftarrow$ somme de tous les sinistres simulés (aucun traité)
2: `charges_nettes_par_annee` $\leftarrow$ appliquer successivement chaque traité de `liste_traites`
3: `cessions_par_annee` $\leftarrow$ `charges_brutes` $-$ `charges_nettes`
4: `prime_reassurance` $\leftarrow$ `compute_premium`(`cessions_par_annee`, `principe_prime`)
5: `charge_finale_par_annee` $\leftarrow$ `charges_nettes` $+$ `prime_reassurance`
6: **for** `vue` $\in$ {`brut`, `cede`, `net_final`} **do**
7: Calculer : `moyenne_charges`, `ecart_type_charges`
8: Calculer : $\mathrm{VaR}_{95\%}$, $\mathrm{VaR}_{99\%}$, $\mathrm{TVaR}_{99\%}$
9: Calculer : `proba_ruine` $= P($`charge` $>$ `capital_disponible`$)$
10: **end for**return métriques des trois vues + indicateurs de rentabilité du traité